如果将 Pessimistic Lock 和 Optimistic Lock 对应到现实生活中来。 Pessimistic Lock 有点像是一位比较悲观（也可以说是未雨绸缪）的人，总是会假设最坏的情况，避免出现问题。 Optimistic Lock 有点像是一位比较乐观的人，总是会假设最好的情况，在要出现问题之前快速解决问题。

什么是 Pessimistic Lock？

Pessimistic Lock 总是假设最坏的情况，认为 Shared Resource 每次被访问的时候就会出现问题（比如 Shared 数据被修改），所以每次在获取资源操作的时候都会上 Lock，这样其他 Thread 想拿到这个资源就会 Blocking 直到 Lock 被上一个持有者释放。也就是说，**Shared Resource 每次只给一个 Thread 使用，其它 Thread Blocking，用完后再把资源转让给其它 Thread**。

像 Java 中 `synchronized` 和 `ReentrantLock` 等 Exclusive Lock 就是 Pessimistic Lock 思想的实现。

```
public void performSynchronisedTask() {
    synchronized (this) {
        // 需要同步的操作
    }
}

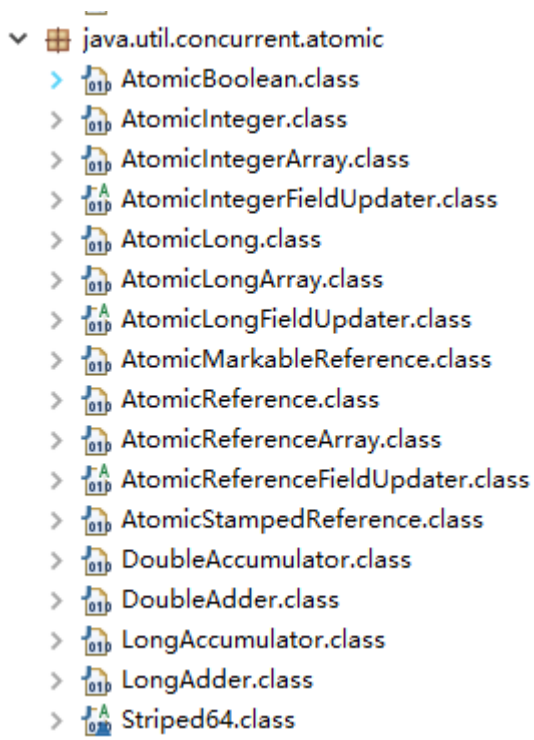
private Lock lock = new ReentrantLock();
lock.lock();
try {
   // 需要同步的操作
} finally {
    lock.unlock();
}
```

High Concurrency 的场景下，激烈的 Lock Contention 会造成 Thread Blocking，大量 Blocking Thread 会导致系统的 Context Switch，增加系统的 Performance Overhead。并且，Pessimistic Lock 还可能会存在 Deadlock 问题（Thread 获得 Lock 的顺序不当时），影响代码的正常运行。

什么是 Optimistic Lock？

Optimistic Lock 总是假设最好的情况，认为 Shared Resource 每次被访问的时候不会出现问题，Thread 可以不停地执行，无需 Locking 也无需等待，只是在提交修改的时候去验证对应的资源（也就是数据）是否被其它 Thread 修改了（具体方法可以使用 Version 机制或 CAS Algorithm）。

在 Java 中 `java.util.concurrent.atomic` Package 下面的 Atomic Variable Class（比如 `AtomicInteger`、`LongAdder`）就是使用了 Optimistic Lock 的一种实现方式 **CAS** 实现的。




```
// LongAdder 在高并发场景下会比 AtomicInteger 和 AtomicLong 的性能更好
// 代价就是会消耗更多的内存空间（空间换时间）
LongAdder sum = new LongAdder();
sum.increment();
```

High Concurrency 的场景下，Optimistic Lock 相比 Pessimistic Lock 来说，不存在 Lock Contention 造成 Thread Blocking，也不会有 Deadlock 问题，在 Performance 上往往会更胜一筹。但是，如果冲突频繁发生（写占比非常多的情况），会频繁失败并 Retry，这样同样会非常影响 Performance，导致 CPU 飙升。

`LongAdder` 会在竞争激烈时把更新分散到多个内部单元，从而降低所有 Thread 争用同一个值的概率，但内部更新仍可能使用 CAS 和 Retry，并没有彻底消除竞争。另外，`sum()` 返回的也不是与 Concurrency 更新保持 Atomic 一致的 Snapshot。

理论上来说：

- Pessimistic Lock 通常多用于写比较多的情况（多写场景，竞争激烈），这样可以避免频繁失败和 Retry 影响 Performance。不过，像 `LongAdder` 这样通过分散竞争来降低 Retry 概率的方案，在只需要累加等特定语义时也可以考虑，仍需根据实际场景衡量。
- Optimistic Lock 通常多用于写比较少的情况（多读场景，竞争较少），这样可以避免频繁 Locking 影响 Performance。不过，Optimistic Lock 主要针对的 Object 是单个 Shared Variable（参考 `java.util.concurrent.atomic` Package 下面的 Atomic Variable Class）。

如何实现 Optimistic Lock？

Optimistic Lock 一般会使用 Version 机制或 CAS Algorithm 实现，CAS Algorithm 相对来说更多一些，这里需要格外注意。

Version 机制

一般是在数据表中加上一个数据 Version `version` Field，表示数据被修改的次数。当数据被修改时，`version` 值会加一。当 Thread A 要更新数据值时，在读取数据的同时也会读取 `version` 值，在提交更新时，若刚才读取到的 version 值为当前 Database 中的 `version` 值相等时才更新，否则 Retry 更新操作，直到更新成功。

举一个简单的例子：假设 Database 中帐户信息表中有一个 version Field，当前值为 1；而当前帐户余额 Field（`balance`）为 $100。

1. 操作员 A 此时将其读出（`version`=1），并从其帐户余额中扣除 $50（$100-$50）。
2. 在操作员 A 操作的过程中，操作员 B 也读入此用户信息（`version`=1），并从其帐户余额中扣除 $20（$100-$20）。
3. 操作员 A 完成了修改工作，将数据 Version（`version`=1），连同帐户扣除后余额（`balance`=$50），提交至 Database 更新，此时由于提交数据版本等于 Database 记录当前版本，数据被更新，Database 记录 `version` 更新为 2。
4. 操作员 B 完成了操作，也将 Version（`version`=1）试图向 Database 提交数据（`balance`=$80），但此时比对 Database 记录版本时发现，操作员 B 提交的数据 Version 为 1，而 Database 记录当前版本为 2，不满足 “ 提交版本必须等于当前版本才能执行更新 ” 的 Optimistic Lock 策略，因此，操作员 B 的提交被驳回。

这样就避免了操作员 B 用基于 `version`=1 的旧数据修改的结果覆盖操作员 A 的操作结果的可能。

CAS Algorithm

CAS 的全称是 **Compare And Swap（比较与交换）**，用于实现 Optimistic Lock，被广泛应用于各大框架中。CAS 的思想很简单，就是用一个预期值和要更新的 Variable 值进行比较，两值相等才会进行更新。

CAS 是一个 Atomic Operation，底层依赖于一条 CPU 的 Atomic Instruction。

> **Atomic Operation** 即最小不可拆分的操作，也就是说操作一旦开始，就不能被打断，直到操作完成。

CAS 涉及到三个 Operand：

- **V**：要更新的 Variable 值(Var)
- **E**：预期值(Expected)
- **N**：拟写入的新值(New)

当且仅当 V 的值等于 E 时，CAS 通过 Atomic 方式用新值 N 来更新 V 的值。如果不等，说明已经有其它 Thread 更新了 V，则 Current Thread 放弃更新。

举一个简单的例子：Thread A 要修改 Variable i 的值为 6，i 原值为 1（V = 1，E=1，N=6，假设不存在 ABA 问题）。

1. i 与 1 进行比较，如果相等， 则说明没被其他 Thread 修改，可以被设置为 6。
2. i 与 1 进行比较，如果不相等，则说明被其他 Thread 修改，Current Thread 放弃更新，CAS 操作失败。

当多个 Thread 同时使用 CAS 操作一个 Variable 时，只有一个会胜出，并成功更新，其余均会失败，但失败的 Thread 并不会被挂起，仅是被告知失败，并且允许再次尝试，当然也允许失败的 Thread 放弃操作。

关于 CAS 的进一步介绍，可以阅读读者写的这篇文章：CAS 详解，其中详细提到了 Java 中 CAS 的实现以及 CAS 存在的一些问题。

总结

本文详细介绍了 Optimistic Lock 和 Pessimistic Lock 的概念以及 Optimistic Lock 常见实现方式：

- Pessimistic Lock 基于悲观的假设，认为 Shared Resource 在每次访问时都会发生冲突，因此在每次操作时都会 Locking。这种 Locking Mechanism 会导致其他 Thread Blocking，直到 Lock 被释放。Java 中的 `synchronized` 和 `ReentrantLock` 是 Pessimistic Lock 的典型实现方式。虽然 Pessimistic Lock 能有效避免数据竞争，但在 High Concurrency 场景下会导致 Thread Blocking、Context Switch 频繁，从而影响系统 Performance，并且还可能引发 Deadlock 问题。
- Optimistic Lock 基于乐观的假设，认为 Shared Resource 在每次访问时不会发生冲突，因此无须 Locking，只需在提交修改时验证数据是否被其他 Thread 修改。Java 中的 `AtomicInteger` 和 `LongAdder` 等 Class 通过 CAS（Compare-And-Swap）Algorithm 实现了 Optimistic Lock。Optimistic Lock 避免了 Thread Blocking 和 Deadlock 问题，在读多写少的场景中 Performance 优越。但在 Write Operation 频繁的情况下，可能会导致大量 Retry 和失败，从而影响 Performance。
- Optimistic Lock 主要通过 Version 机制或 CAS Algorithm 实现。 Version 机制通过比较 Version 确保数据一致性，而 CAS 通过硬件 Instruction 实现 Atomic Operation，直接比较和交换 Variable 值。

Pessimistic Lock 和 Optimistic Lock 各有优缺点，适用于不同的应用场景。在实际开发中，选择合适的 Locking Mechanism 能够有效提升系统的 Concurrency Performance 和稳定性。

参考

- 《Java Concurrent Programming 核心 78 讲》
- 通俗易懂 Pessimistic Lock、Optimistic Lock、Reentrant Lock、Spin Lock、Biased Lock、轻量/Heavyweight Lock、Read-Write Lock、各种 Lock 及其 Java 实现！：https://zhuanlan.zhihu.com/p/71156910